02624 Inverse Problems and Imaging — Homework 2

Jeppe Klitgaard <s250250@dtu.dk>
Date: 2025-10-24

Contents

Introduction 0
1) 1D Heat Problem 1
  1.a) Deriving the Ordinary Tikhonov Solution 1
  1.b) Analytical and Sampled Bayesian Estimation of Initial Temperature 2
  1.c) Reduced Observational Region 3
2) Abel Problem and H^1-Regularisation 4
  2.a) Show that $x(s) \in H^1(0,1)$ 4
  2.b) Uniqueness and Existence of Generalised Tikhonov Solution 4
  2.*) Closed Form Solution for Generalised Tikhonov 5
  2.c) Ordinary Tikhonov and Discrepancy Principle 5
  2.d) Generalised Tikhonov and Discrepancy Principle 6
A) Appendix 8
  A.1) Code preamble 8
  A.2) Code for Section 1.b 9
  A.3) Code for Section 1.c 12
  A.4) Code for Section 2.c 17
  A.5) Code for Section 2.d 23
  A.6) Google Gemini 2.5 Pro Review Comments 26
Bibliography 34

Introduction

This report presents solutions to Homework 2 for the course 02624 Inverse Problems and Imaging.

GitHub Copilot was used for tab completions in the development of the code associated with this report. Google Gemini 2.5 Pro was used for proof-reading and improvement suggestions on the final draft of the report. The prompt input and output is included in appendix A.6. Only a few of the suggestions were adopted, primarily related to grammar or repeated words.

The code used to generate experiments and figures can be found in the appendix.

1) 1D Heat Problem

We consider the 1D backwards heat problem on the interval $[0, \pi]$ over time period $t \in (0, T \coloneqq 0.1]$ modelled by the operator K defined such that $y(t) = Kx(t)$, corresponding to the problem:

$$\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial x^2} = 0, \qquad 0 < x < \pi, \quad t \in (0, T] \tag{1}$$

$$u(0,t) = u(\pi, t) = 0, \qquad u(x, 0) = f(x) \tag{2}$$

The operator K may be discretised on equidistant grids of $N = 128$ points in both space and time to obtain a matrix $A \in \mathbb{R}^{N \times N}$ such that $y = Ax$ approximates the continuous case.

The problem is considered in a Bayesian framework with Gaussian noise as given by:

$$Y = Ax + E \tag{3}$$

Where x is a realisation of the prior distribution $X \sim \mathcal{N}(0, \gamma^2 I)$ with $\gamma \coloneqq 10^{-1}$ and E is the Gaussian noise distributed as $E \sim \mathcal{N}(0, \sigma^2 I)$ where $\sigma \coloneqq 10^{-3}$. As such, we find that observations are determined by the realisation of the noise, e, as:

$$y_{\text{obs}} = y + e \tag{4}$$

Thus, we obtain probability densities:

$$\begin{aligned} \pi_{\text{pr}}(x) &\propto \exp\left(-\frac{x^2}{2\gamma^2}\right) \\ \pi(y_{\text{obs}}|x) &\propto \exp\left(-\frac{(y_{\text{obs}} - Ax)^2}{2\sigma^2}\right) \\ \pi(y_{\text{obs}}) &= \int \pi(y_{\text{obs}}|x)\pi_{\text{pr}}(x)\,\mathrm{d}x \propto 1 \end{aligned} \tag{5}$$

From which we may obtain the posterior distribution through Bayes' theorem:

$$\begin{aligned} \pi_{\text{post}} = \pi(x|y_{\text{obs}}) &= \frac{\pi(y_{\text{obs}}|x)\pi_{\text{pr}}(x)}{\pi(y_{\text{obs}})} \\ &\propto \exp\left(-\frac{1}{2}\left(\frac{\|y_{\text{obs}} - Ax\|_2^2}{\sigma^2} + \frac{\|x\|_2^2}{\gamma^2}\right)\right) \\ &\propto \exp\left(-\frac{1}{2}(x - \bar{x})^{\mathsf{T}}\Gamma_{\text{post}}^{-1}(x - \bar{x})\right) \end{aligned} \tag{6}$$

Where the last relation proposes that the posterior is also Gaussian with mean $\bar{x}$ and covariance Γ_{post}.

1.a) Deriving the Ordinary Tikhonov Solution

We may verify that (6) holds and in doing so obtain expressions for $\bar{x}$ and Γ_{post} by the following treatment:

$$\begin{aligned} \exp\left(-\frac{1}{2}(x - \bar{x})^{\mathsf{T}}\Gamma_{\text{post}}^{-1}(x - \bar{x})\right) &\propto \exp\left(-\frac{1}{2}\left(\frac{\|y_{\text{obs}} - Ax\|_2^2}{\sigma^2} + \frac{\|x\|_2^2}{\gamma^2}\right)\right) \\ &= \exp\left(-\frac{1}{2}\left[\frac{(y_{\text{obs}} - Ax)^{\mathsf{T}}(y_{\text{obs}} - Ax)}{\sigma^2} + \frac{x^{\mathsf{T}}x}{\gamma^2}\right]\right) \\ &= \exp\left(-\frac{1}{2}\left[\frac{1}{\sigma^2}\left(y_{\text{obs}}^{\mathsf{T}}y_{\text{obs}} - y_{\text{obs}}^{\mathsf{T}}Ax - x^{\mathsf{T}}A^{\mathsf{T}}y_{\text{obs}} + x^{\mathsf{T}}A^{\mathsf{T}}Ax\right) + \frac{x^{\mathsf{T}}x}{\gamma^2}\right]\right) \\ &= \exp\left(-\frac{1}{2}\left[\underbrace{x^{\mathsf{T}}\left(\frac{A^{\mathsf{T}}A}{\sigma^2} + \frac{I}{\gamma^2}\right)x}_{\text{Quadratic}} - \underbrace{2x^{\mathsf{T}}\frac{A^{\mathsf{T}}}{\sigma^2}y_{\text{obs}}}_{\text{Linear}} + \underbrace{\frac{y_{\text{obs}}^{\mathsf{T}}y_{\text{obs}}}{\sigma^2}}_{\text{Constant}}\right]\right) \end{aligned} \tag{7}$$

Which by comparison against the standard form of the Gaussian distribution given in (6) reveals:

$$\begin{aligned} \Gamma_{\text{post}}^{-1} &= \frac{A^{\mathsf{T}}A}{\sigma^2} + \frac{I}{\gamma^2} \\ \bar{x} &= \Gamma_{\text{post}}A^{\mathsf{T}}\frac{y_{\text{obs}}}{\sigma^2} \end{aligned} \tag{8}$$

We may now invert $\Gamma_{\text{post}}^{-1}$ to obtain Γ_{post} using the Woodbury matrix identity [1, Eq. 159] to find:

$$\begin{aligned}\Gamma_{\text{post}} &= \gamma^2 I - \gamma^2 \frac{A^\mathsf{T}}{\sigma}\left(I + \frac{A}{\sigma}\gamma^2\frac{A^\mathsf{T}}{\sigma}\right)^{-1}\frac{A}{\sigma}\gamma^2 \\ &= \gamma^2 I - \gamma^4 A^\mathsf{T}(\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}A\end{aligned} \tag{9}$$

Using which we obtain the final expression for the posterior mean as:

$$\begin{aligned}\bar{x} &= \gamma^2\frac{A^\mathsf{T}}{\sigma^2}y_{\text{obs}} - \gamma^4 A^\mathsf{T}(\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}A\frac{A^\mathsf{T}}{\sigma^2}y_{\text{obs}} \\ &= \frac{\gamma^2}{\sigma^2}A^\mathsf{T}\Big(I - (\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}\gamma^2 AA^\mathsf{T}\Big)y_{\text{obs}} \\ &= \frac{\gamma^2}{\sigma^2}A^\mathsf{T}\Big(I - (\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}((\sigma^2 + \gamma^2 AA^\mathsf{T}) - \sigma^2 I)\Big)y_{\text{obs}} \\ &= \frac{\gamma^2}{\sigma^2}A^\mathsf{T}\Big(I - I + \sigma^2(\sigma^2 I + \gamma^2 AA^\mathsf{T})^{-1}\Big)y_{\text{obs}} \\ &= \gamma^2 A^\mathsf{T}(\sigma^2 I + \gamma^2 AA^\mathsf{T})^{-1}y_{\text{obs}}\end{aligned} \tag{10}$$

We seek to put this into the standard Tikhonov form [2, Eq. 2.18] for a solution $\bar{x} = R_\alpha y_{\text{obs}}$ for the discrete operator A:

$$R_\alpha \coloneqq (\alpha I + A^\mathsf{T}A)^{-1}A^\mathsf{T} \tag{11}$$

We do this by manipulating (10) and introducing $\alpha \coloneqq \sigma^2/\gamma^2$:

$$\begin{aligned}\bar{x} &= \gamma^2 A^\mathsf{T}(\sigma^2 I + \gamma^2 AA^\mathsf{T})^{-1}y_{\text{obs}} \\ \bar{x} &= A^\mathsf{T}(\alpha I + AA^\mathsf{T})^{-1}y_{\text{obs}} \\ (\alpha I + A^\mathsf{T}A)\bar{x} &= (\alpha I + A^\mathsf{T}A)A^\mathsf{T}(\alpha I + AA^\mathsf{T})^{-1}y_{\text{obs}} \\ (\alpha I + A^\mathsf{T}A)\bar{x} &= A^\mathsf{T}(\alpha I + AA^\mathsf{T})(\alpha I + AA^\mathsf{T})^{-1}y_{\text{obs}} \\ (\alpha I + A^\mathsf{T}A)\bar{x} &= A^\mathsf{T}y_{\text{obs}} \\ \bar{x} &= (\alpha I + A^\mathsf{T}A)^{-1}A^\mathsf{T}y_{\text{obs}}\end{aligned} \tag{12}$$

As such, we have recovered the standard Tikhonov solution form as desired.

Lastly, we seek to show that the posterior covariance, Γ_{post}, satisfies $\Gamma_{\text{post}} < \gamma^2 I$, which by inspection of (9) is equivalent to showing that the second term is positive definite, which we also note to be symmetric by inspection:

$$\cancel{\gamma^4} A^\mathsf{T}(\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}A \succ 0 \tag{13}$$

Where $\gamma^4 > 0$ by construction and may be set to unity without loss of generality. We leverage the quadratic form definition of positive definiteness [1, Eq. 439] to show that for any non-zero vector $y \in \mathbb{R}^N$:

$$y^\mathsf{T}A^\mathsf{T}(\sigma^2 + \gamma^2 AA^\mathsf{T})^{-1}Ay = z^\mathsf{T}B^{-1}z \tag{14}$$

Where we have defined $z \coloneqq Ay$ and $B \coloneqq (\sigma^2 + \gamma^2 AA^\mathsf{T})$ and seek to leverage the property that the inverse of a matrix M is positive definite if M itself is positive definite [1, §9.6.4]:

$$z^\mathsf{T}Bz = \sigma^2 z^\mathsf{T}z + \gamma^2 z^\mathsf{T}AA^\mathsf{T}z = \sigma^2\|z\|_2^2 + \gamma^2\left\|A^\mathsf{T}z\right\|_2^2 > 0 \tag{15}$$

As such, we find that B is positive definite, thus implying B^{-1} to also be positive definite, which in turn requires that $\Gamma_{\text{post}} < \gamma^2 I$ by the construction above.

This result is consistent with our intuition that incorporation of observations into the posterior distribution should only reduce uncertainty in the estimate when compared against the prior distribution.

1.b) Analytical and Sampled Bayesian Estimation of Initial Temperature

We proceed to probe the initial temperature estimation of the problem described above on an equidistant grid of $N = 128$ points over the domain $s \in [0, \pi]$ by drawing N_{samples} samples from the posterior distribution $\pi_{\text{post}}(x)$ given in (6) from which the sample mean and variances are computed across the domain.

Additionally, the posterior mean and variances may be computed directly using the analytical expressions found in (9) and (10).

The results of both approaches are shown in Figure 1, in which we observe close agreement between the analytical and sampled approaches, with both approximating the true initial temperature profile. The effect

of finite sampling is particularly visible on the credibility interval of the sampled mean as observed in the left panel of the figure, which may be improved by increasing N_{samples} as seen in the right panel.

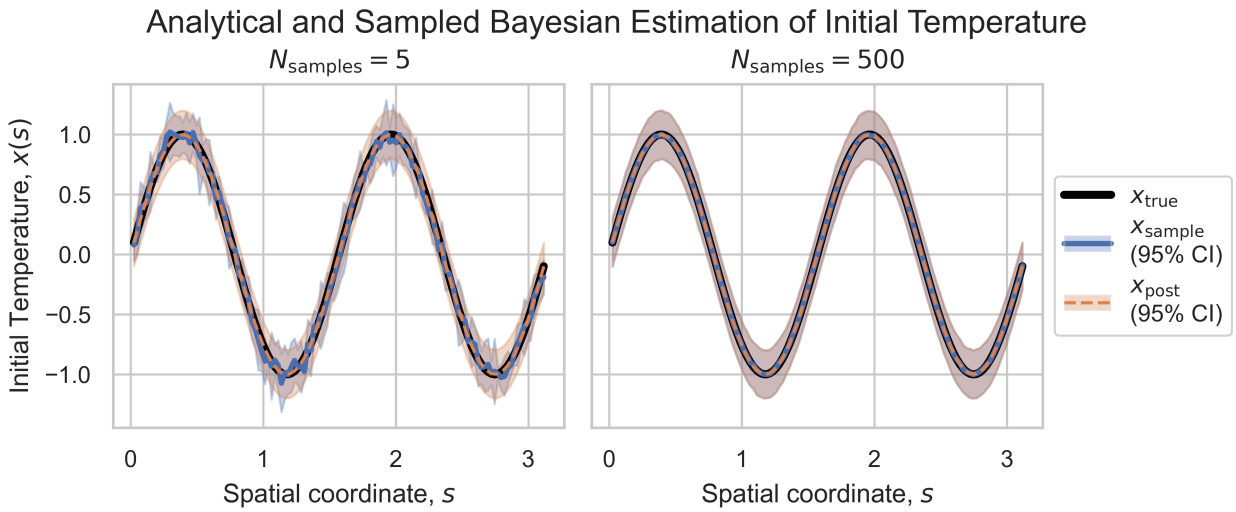


Figure 1: Analytical and sampled Bayesian estimation of the initial temperature $x(s)$ from noisy observations y_{obs} at time $T = 0.1$ with noise level $\sigma = 10^{-3}$ and prior standard deviation $\gamma = 10^{-1}$. The black line indicates the true initial temperature.

1.c) Reduced Observational Region

We now consider the case in which only the interval $\left(\frac{1}{4}\pi, \frac{3}{4}\pi\right)$ of the spatial domain is observed, which limits the amount of information available for the inversion. This is done by only including the rows of A and entries of y_{obs} corresponding to this region when computing the posterior mean and covariance.

Given our heat problem is defined with unit diffusivity, the terminal time T alone determines the degree of ill-posedness of the problem, with shorter terminal times ordinarily resulting in a less ill-posed problem. However, by limiting the observation region, a longer terminal time allows more information to propagate into the observed region from the unobserved regions, which may improve the inversion results in the unobserved regions.

For this reason, we investigate the inversion for a longer terminal time of $T = 0.1$ as compared to the shorter time of $T = 0.005$ as seen in Figure 2.

We may compare the left panel of Figure 2 against Figure 1 to observe the degradation in inversion quality due to the limited observation region, where notably the uncertainty in the unobserved regions increases and the estimate is shifted towards the prior mean. This effect is more pronounced in the right panel corresponding to a shorter terminal time of $T = 0.005$, in which the uncertainty away from the observations approaches the level of the prior distribution and the estimate reverts to that of the prior.

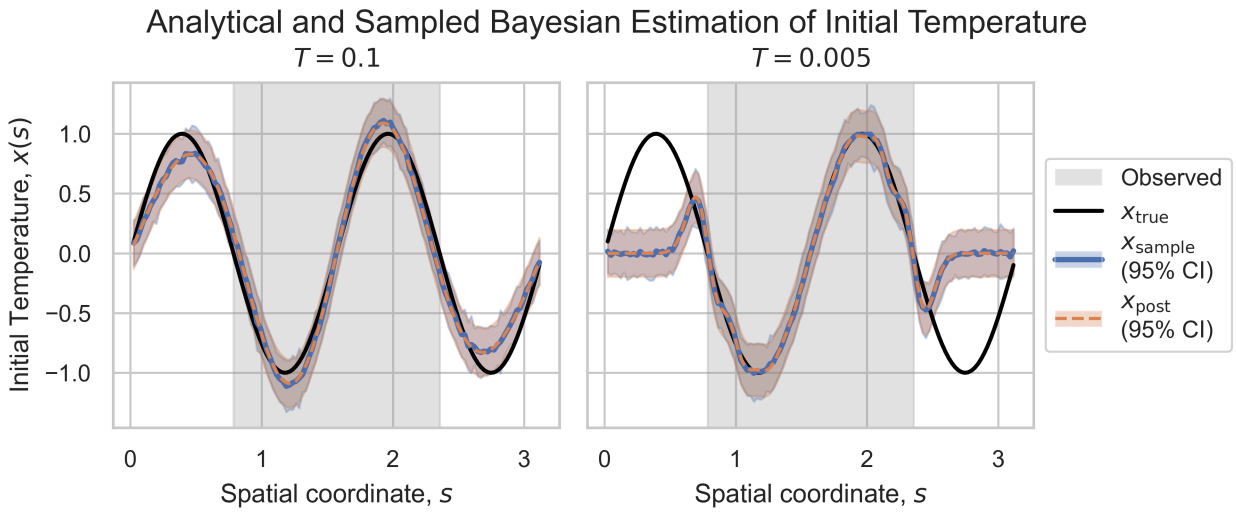


Figure 2: Analytical and sampled Bayesian estimation of the initial temperature $x(s)$ from noisy observations y_{obs} at times $T = 0.1$ (left) and $T = 0.005$ (right) with noise level $\sigma = 10^{-3}$ and prior standard deviation $\gamma = 10^{-1}$. Only the region $\left(\frac{1}{4}\pi, \frac{3}{4}\pi\right)$ is observed. The black line indicates the true initial temperature.

2) Abel Problem and H^1-Regularisation

We consider the Abel problem corresponding to the integral operator K defined such that $y(t) = Kx(t)$, where:

$$Kx(t) = \int_0^1 k(t,s)x(s)\,\mathrm{d}s, \qquad k(t,s) = \begin{cases} \frac{1}{\sqrt{t-s}}, & s < t \\ 0, & s > t \end{cases}, \qquad x(s) = s^{3/4} - 1 \tag{16}$$

We again discretise the operator K by a matrix $A \in \mathbb{R}^{N\times N}$ where $N = 1024$. We introduce a Gaussian measurement noise of 10% relative to the norm of the noiseless measurements to produce noisy observations y^δ from $y = Ax$. Lastly, we let D denote a finite difference matrix such that Dx approximates the first derivative of x.

2.a) Show that $x(s) \in H^1(0,1)$

In order to show that $x(s) = s^{\{\frac{3}{4}\}} - 1$ lies in the Sobolev space $H^1(0,1)$, we need to demonstrate that both $x(s)$ and its weak derivative $x'(s)$ are square-integrable over the interval. We first consider $x(s)$:

$$\begin{aligned} \int_0^1 |x(s)|^2\,\mathrm{d}s &= \int_0^1 \left(s^{\frac{3}{4}} - 1\right)^2 \mathrm{d}s = \int_0^1 \left(s^{\frac{3}{2}} - 2s^{\frac{3}{4}} + 1\right)\mathrm{d}s \\ &= \left[\left(\frac{2}{5}\right)s^{\frac{5}{2}} - 2\cdot\frac{4}{7}s^{\frac{7}{4}} + s\right]_0^1 = \frac{14}{35} - \frac{40}{35} + \frac{35}{35} = \frac{9}{35} < \infty \end{aligned} \tag{17}$$

Thus proving that $x(s)$ is square-integrable. Next, we compute the weak derivative, here equivalent to the classical derivative due to the smoothness of $x(s)$:

$$x'(s) = \frac{3}{4}s^{-\frac{1}{4}} \tag{18}$$

Which we show to be square-integrable as well:

$$\int_0^1 |x'(s)|^2\,\mathrm{d}s = \int_0^1 \left(\frac{3}{4}s^{-\frac{1}{4}}\right)^2 \mathrm{d}s = \frac{9}{16}\int_0^1 s^{-\frac{1}{2}}\,\mathrm{d}s = \frac{9}{16}\left[2s^{\frac{1}{2}}\right]_0^1 = \frac{9}{8} < \infty \tag{19}$$

Consequently, we conclude that $x(s) \in H^1(0,1)$ as implied by $x(s) \in L^2(0,1)$ and $x'(s) \in L^2(0,1)$.

2.b) Uniqueness and Existence of Generalised Tikhonov Solution

By approximating the H^1 norm by $\|x\|_2^2 + \|Dx\|_2^2$, we obtain the following optimisation problem for the generalised Tikhonov solution:

$$x_\alpha = \underset{x\in H^1(0,1)}{\arg\min} \left\|Ax - y^\delta\right\|_2^2 + \alpha(\|x\|_2^2 + \|Dx\|_2^2) \tag{20}$$

We let the objective function be denoted $f(x)$ and seek to prove that a unique solution x_α exists for any $\alpha > 0$.

To prove existence, we seek to use Weierstrass Extreme Value Theorem [3, S. 16], which states that a continuous function defined on a compact set attains a global minimum on that set. Given that the objective function is continuous by inspection, we only need to prove that the sublevel sets $S_c = \{x \in \mathbb{R}^n : f(x) \le c\}$ are bounded, which is implied by the coercivity of the objective function:

$$\left\|Ax - y^\delta\right\|_2^2 + \alpha(\|x\|_2^2 + \|Dx\|_2^2) \to +\infty \text{ as } \|x\|_2 \to +\infty \tag{21}$$

Where the derivative and residual terms are non-negative by construction, thus reducing the problem to showing that $\alpha\|x\|_2^2$ is coercive, which is trivially true for any $\alpha > 0$. This allows us to restrict our search for a minimiser to a sublevel set which is compact, thus allowing us to apply the Extreme Value Theorem to conclude that a global minimiser exists on an arbitrarily chosen sublevel set S_c.

Uniqueness may be determined by considering the convexity of the objective function. Any norm is convex by construction, and raising to a power $p \ge 1$ preserves convexity, thus implying that the squared norms are convex.

Non-negative scaling likewise preserves convexity, thus preserving the convexity of the second term $\alpha(\|x\|_2^2 + \|Dx\|_2^2)$ for $\alpha > 0$. Lastly, the sum of convex functions is itself convex, thus implying that the entire objective function is convex. To guarantee uniqueness, we require *strict* convexity, which may be guaranteed by considering the Hessian of $\|x\|_2^2$:

$$\nabla^2\|x\|_2^2 = 2I \succ 0 \tag{22}$$

Thus implying that $\|x\|_2^2$ is strictly convex by the second order condition.

This approach relies heavily on the known convexity properties of various functions and operations as taught by the 02611 Optimisation for Data Science course. As an alternative, we may rely on the second order condition to prove strict convexity of the entire objective function by considering its Hessian:

$$\nabla^2 \left[\left\| Ax - y^\delta \right\|_2^2 + \alpha(\|x\|_2^2 + \|Dx\|_2^2) \right] = 2A^\mathsf{T}A + 2\alpha I + 2\alpha D^\mathsf{T}D \tag{23}$$

Where $A^\mathsf{T}A$ and $D^\mathsf{T}D$ are both positive semi-definite by the same treatment used in Section 1.a:

$$y^\mathsf{T}B^\mathsf{T}By = \|By\|_2^2 \geq 0 \qquad \Leftrightarrow \qquad B^\mathsf{T}B \succcurlyeq 0 \tag{24}$$

Where $y \in \mathbb{R}^N$ and B is either A or D, thus implying:

$$\nabla^2 \left[\left\| Ax - y^\delta \right\|_2^2 + \alpha(\|x\|_2^2 + \|Dx\|_2^2) \right] = 2A^\mathsf{T}A + 2\alpha I + 2\alpha D^\mathsf{T}D \succ 0 \tag{25}$$

As such, we conclude that the objective function is strictly convex and thus admits a unique minimiser for any $\alpha > 0$.

2.*) Closed Form Solution for Generalised Tikhonov

In order to compute the Tikhonov solutions for the subsequent sections, we will first derive a closed form solution to the generalised Tikhonov problem from (20), to which end we compute the gradient and set it to zero to find the minimiser:

$$\begin{aligned} 0 &= \nabla \Big(\left\| Ax - y^\delta \right\|_2^2 + \alpha(\|x\|_2^2 + \|Dx\|_2^2) \Big) \\ &= \nabla \Big(\big(Ax - y^\delta\big)^\mathsf{T} \big(Ax - y^\delta\big) + \alpha(x^\mathsf{T}x + x^\mathsf{T}D^\mathsf{T}Dx) \Big) \\ &= 2A^\mathsf{T}Ax - 2A^\mathsf{T}y^\delta + 2\alpha x + 2\alpha D^\mathsf{T}Dx \\ &= (A^\mathsf{T}A + \alpha I + \alpha D^\mathsf{T}D)x - A^\mathsf{T}y^\delta \end{aligned} \tag{26}$$

Which we may solve for x to obtain the closed form solution for the generalised Tikhonov problem:

$$x_\alpha = \left(A^\mathsf{T}A + \alpha(I + D^\mathsf{T}D)\right)^{-1} A^\mathsf{T}y^\delta \tag{27}$$

Where the ordinary Tikhonov solution may be obtained by setting $D = 0$.

$$x_\alpha = \left(A^\mathsf{T}A + \alpha I\right)^{-1} A^\mathsf{T}y^\delta \tag{28}$$

These closed-form solutions are used in the computational experiments of the subsequent sections.

2.c) Ordinary Tikhonov and Discrepancy Principle

We seek to find an optimal regularisation parameter $\alpha > 0$ by applying the Morozov discrepancy principle to the ordinary Tikhonov solution, (28). Additionally, we assume that an estimate of the noise level δ is available, in which case the discrepancy principle requires that we find an α such that [4, §5.2]:

$$\left\| Kx_\alpha - y^\delta \right\| \approx \nu\delta \tag{29}$$

Where $\nu \geq 1$, here chosen as $\nu = 1.05$, is a safety factor chosen to produce a conservative estimate of the regularisation parameter. We find such an α_{DP} by performing a search across 200 linearly spaced values of the regularisation parameter $\alpha \in [0, 0.2]$ and selecting the value that most closely satisfies the discrepancy principle to obtain Figure 3, in which we also display the error norm $\|x_\alpha - x\|_2$ to gauge the performance of the discrepancy principle.

We find that the discrepancy principle provides an estimated regularisation parameter very close the optimal value. Additionally, we observe in the right panel that the error does not depend strongly on the choice of α within the neighbourhood of the optimal value, thus making the discrepancy principle a robust method for choosing the regularisation parameter in this case.

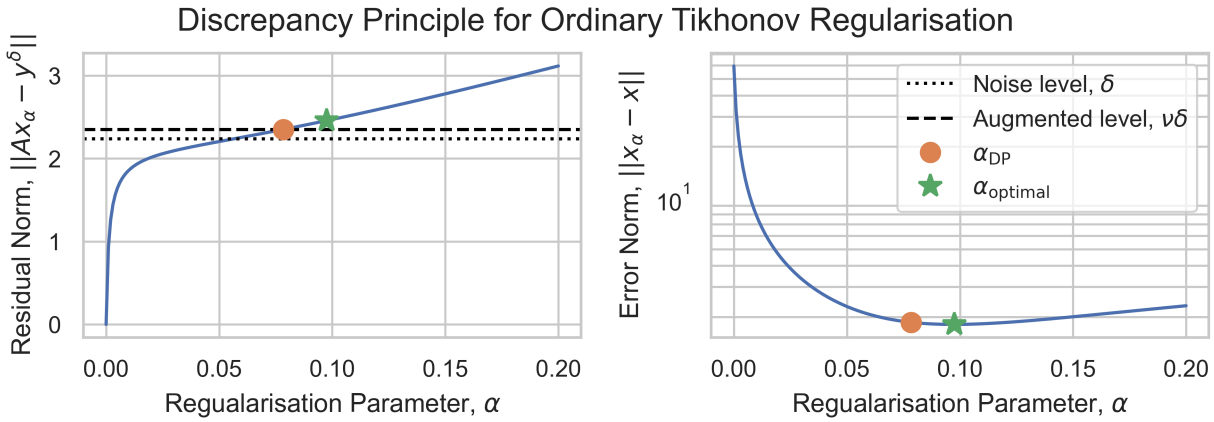


Figure 3: Application of the Morozov discrepancy principle to determine an optimal regularisation parameter α for the ordinary Tikhonov solution of the Abel problem. The left panel contains the residual norm $\left\|Ax_\alpha - y^\delta\right\|_2$ while the right panel contains the error norm $\left\|x_\alpha - x\right\|_2$. The horizontal dashed line indicates the target discrepancy level $\nu\delta$.

2.d) Generalised Tikhonov and Discrepancy Principle

We repeat the procedure of Section 2.c for the generalised Tikhonov solution, where we let D denote a central difference first order differentiation matrix such that Dx approximates the first derivative of x. At the boundaries, the first order forward and backward difference schemes are used instead:

$$D = \frac{1}{h}\begin{bmatrix} -1 & 1 & & & \\ -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & \ddots & \ddots & \ddots & \\ & & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & & -1 & 1 \end{bmatrix}, \qquad h = \frac{1}{N} \tag{30}$$

We again perform the search undertaken in Section 2.d, now using the generalised Tikhonov solution from (27) to obtain Figure 4, where notably the search interval has been changed to $\alpha \in [0, 0.015]$ to better capture the behaviour of the generalised solution. When compared against the ordinary Tikhonov solution of Section 2.c, we find that a lower degree of regularisation is required on the norm of x when the norm of its derivative is also penalised, which is consistent with our intuition given the high degree of smoothness of the true solution, which is shown alongside the solutions below in Figure 5.

Notably, the generalised Tikhonov solution is able to provide a better reconstruction as evidenced by the lower error norms observed in the right panel of Figure 4 when compared against Figure 3. Interestingly, the introduction of the safety factor $\nu = 1.05$ appears to negatively impact the performance of the discrepancy principle in this case, as can be seen in the left panel of Figure 4, where the unaugmented noise level matches the optimal solution almost exactly.

Were we to remove the safety factor for both the ordinary and generalised Tikhonov solutions however, we observe from Figure 3 that the ordinary Tikhonov solution would suffer from under-regularisation as a result. As such, the choice of safety factor appears to require a heuristic or intuition-based approach in and of itself, though notably the impact of the choice of ν will not be known without access to the true solution.

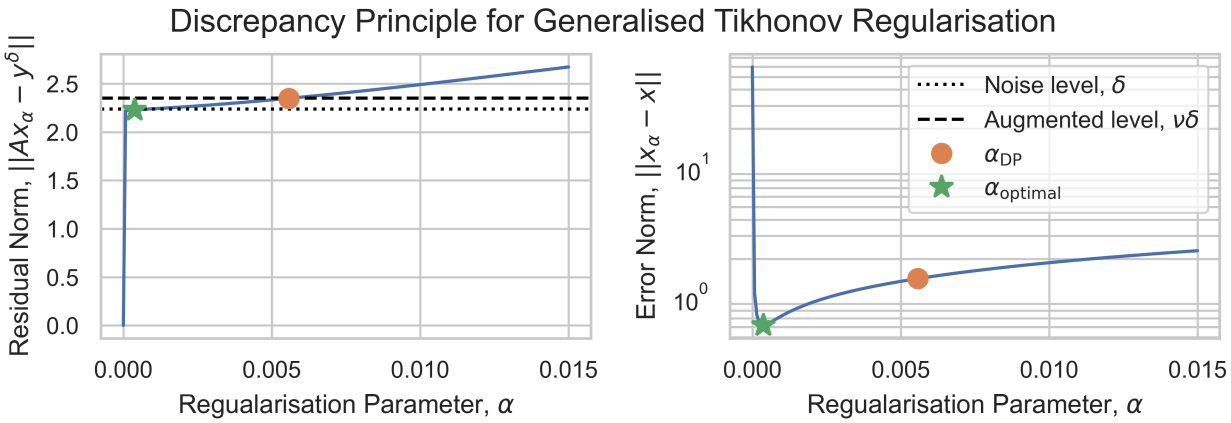


Figure 4: Morozov discrepancy principle applied to the generalised Tikhonov solution of the Abel problem.

Lastly, we compare the reconstructions obtained by both the ordinary and generalised Tikhonov solutions using the regularisation parameters obtained by the discrepancy principle in Figure 5, which reveals that the generalised solution is able to capture the smoothness of the underlying problem to significantly reduce the noise in the reconstruction when compared against the ordinary solution. Interestingly, both solutions appear biased at the left side of the domain, whereas only the generalised solution appears biased at the right side.

It would be interesting to further investigate how the performance of a generalised Tikhonov solution could be improved by introducing separate regularisation parameters for the norm of x and its derivative.

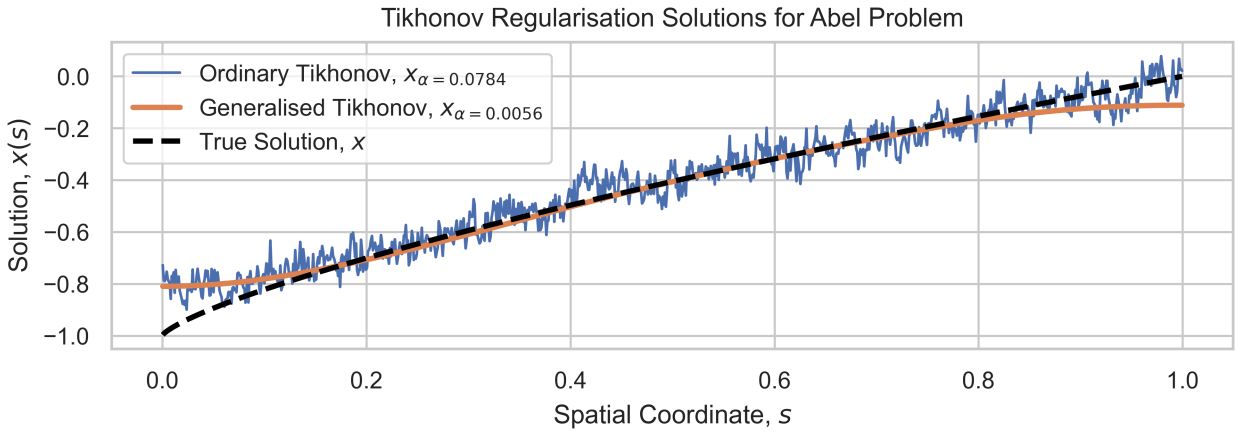


Figure 5: Comparison of the ordinary and generalised Tikhonov solutions of the Abel problem using regularisation parameters obtained by the Morozov discrepancy principle.

A) Appendix

A.1) Code preamble

Python

```python
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import numpy as np
from scipy.fft import dst, idst
import matplotlib.pyplot as plt
import numpy.typing as npt
import matplotlib as mpl
from tqdm.auto import tqdm

# ---
OUTPUT_DIR = Path("./output")
OUTPUT_DIR.mkdir(exist_ok=True)

mm = 1 / 25.4  # mm in inches
PAPER_WIDTH_IN = (210 - 15 - 25) * mm

EXPORT_DPI = 600

sns.set_theme(context="paper", style="whitegrid")

# ---
# Based on: https://stackoverflow.com/a/29261252/5036246
def sci_notation(number, sig_fig=2, latex_style=True, omit_coeff_if_unity=True):
    ret_string = "{0:.{1:d}e}".format(number, sig_fig)
    a, b = ret_string.split("e")

    # Calculate factor part
    factor = ""
    joiner = ""
    if omit_coeff_if_unity and float(a) == 1.0:
        factor = ""
        joiner = ""
    else:
        factor = a
        joiner = "×"

    # Calculate exponent part
    b = int(b)
    exponent = ""
    if latex_style:
        exponent = f"10^{{{str(b)}}}"
    else:
        exponent = f"10^{str(b)}"

```

```
    return factor + joiner + exponent
```

A.2) Code for Section 1.b

Python

```
def forward_heat_operator(N_int: int, T: float) -> npt.NDArray:
    """
    Creates the discrete forward operator A for the 1D heat equation.

    The operator maps the initial state u(s, 0) to the final state u(s, T).
    It is constructed using the Discrete Sine Transform (DST).

    Note that the spatial domain is assumed to be (0, π).

    Arguments:
        N_int: The number of internal grid points.
        T: The final time.

    Returns:
        Matrix A of shape (N_int, N_int) that can be applied to the _internal_ state
        vector.
    """

    k = np.arange(N_int) + 1  # Wavenumbers
    decay_factors = np.exp(-(k**2) * T)  # Decay factors for each mode

    A = np.zeros((N_int, N_int))
    for j in range(N_int):
        # Basis vector
        e_j = np.zeros(N_int)
        e_j[j] = 1.0

        # Type I DST corresponding to sine series expansion
        coeffs = dst(e_j, type=1, norm="ortho")
        decayed_coeffs = decay_factors * coeffs
        col_j = idst(decayed_coeffs, type=1, norm="ortho")

        A[:, j] = col_j

    return A


# Parameters
N = 128
N_int = N - 2
T = 0.1
a = 0
b = np.pi

# Get spatial discretisation
s_grid = np.linspace(a, b, N, endpoint=True)
s_internal = s_grid[1:-1]  # Internal points only

```

```
48
49  def analytical_1(s: npt.NDArray) -> npt.NDArray:
50      return np.sin(4 * s)
51
52
53  x_initial = analytical_1(s_grid)
54  x_initial_internal = x_initial[1:-1]
55
56  # Get operator
57  A = forward_heat_operator(N_int, T)
58  y_final_numeric_internal = A @ x_initial_internal
59
60  # ---
61  np.random.seed(42)
62
63  gamma = 1e-1
64  sigma = 1e-3
65
66  N_simulations = 10
67  I = np.eye(*A.shape)
68
69  fig, (ax_coarse, ax_fine) = plt.subplots(
70      1, 2, figsize=(PAPER_WIDTH_IN, 3), sharey=True, sharex=True
71  )
72  for N_simulations, ax in [(5, ax_coarse), (500, ax_fine)]:
73      posterior_covariance = (
74          gamma**2 * I
75          + gamma**4 * A.T @ np.linalg.inv(gamma**2 * A @ A.T + sigma**2 * I) @ A
76      )
77      posterior_mean = (
78          gamma**2
79          * A.T
80          @ np.linalg.inv(gamma**2 * A @ A.T + sigma**2 * I)
81          @ y_final_numeric_internal
82      )
83      posterior_variance = np.diag(posterior_covariance)
84      x_sample = np.random.multivariate_normal(
85          posterior_mean, posterior_covariance, size=N_simulations
86      )
87
88      sample_mean = np.mean(x_sample, axis=0)
89      sample_variance = np.var(x_sample, axis=0)
90
91      ax.plot(
92          s_internal,
93          x_initial_internal,
94          color="black",
95          linewidth=3.0,
96          label="Initial condition",
97      )
```

```
    ax.plot(s_internal, sample_mean, color="C0", lw=2.0, label="Sample mean")
    ax.plot(s_internal, posterior_mean, color="C1", ls="--", label="Posterior mean")

    # Plot 95% credible intervals
    sample_upper_bound = sample_mean + 1.96 * np.sqrt(sample_variance)
    sample_lower_bound = sample_mean - 1.96 * np.sqrt(sample_variance)
    posterior_upper_bound = posterior_mean + 1.96 * np.sqrt(posterior_variance)
    posterior_lower_bound = posterior_mean - 1.96 * np.sqrt(posterior_variance)
    ax.fill_between(
        s_internal, sample_lower_bound, sample_upper_bound, color="C0", alpha=0.3
    )
    ax.fill_between(
        s_internal, posterior_lower_bound, posterior_upper_bound, color="C1",
        alpha=0.3
    )

    ax.set_title(f"$N_\\mathrm{{samples}} = {N_simulations}$")

legend_elements = [
    mpl.lines.Line2D([0], [0], color="black", linewidth=3.0),
    (
        mpl.lines.Line2D([0], [0], color="C0", linewidth=2.0, ls="-"),
        mpl.patches.Patch(facecolor="C0", alpha=0.3),
    ),
    (
        mpl.lines.Line2D([0], [0], color="C1", ls="--"),
        mpl.patches.Patch(facecolor="C1", alpha=0.3),
    ),
]

legend_labels = [
    "$x_\\mathrm{true}$",
    "$x_\\mathrm{sample}$\n(95% CI)",
    "$x_\\mathrm{post}$\n(95% CI)",
]

ax.legend(legend_elements, legend_labels, loc="center left", bbox_to_anchor=(1,
0.5))

ax_coarse.set_xlabel("Spatial coordinate, $s$")
ax_fine.set_xlabel("Spatial coordinate, $s$")
ax_coarse.set_ylabel("Initial Temperature, $x(s)$")

fig.suptitle(
    "Analytical and Sampled Bayesian Estimation of Initial Temperature",
    fontsize=12,
    y=0.92,
)
fig.tight_layout()
plt.savefig(OUTPUT_DIR / "1_b.png", dpi=EXPORT_DPI, bbox_inches="tight")

```

```
# ---
# Visualise temperature progression
for T_final in np.linspace(0.001, 0.25, 5):
    A_original = forward_heat_operator(N_int, T_final)
    y = A_original @ x_initial_internal

    plt.plot(s_internal, y, label=f"T={T_final:.3f}")
```

A.3) Code for Section 1.c

Python

```
# Parameters
N = 128
N_int = N - 2
T = 0.1
a = 0
b = np.pi

gamma = 1e-1
sigma = 1e-3

# Get spatial discretisation
s_grid = np.linspace(a, b, N, endpoint=True)
s_internal = s_grid[1:-1]  # Internal points only


def analytical_1(s: npt.NDArray) -> npt.NDArray:
    return np.sin(4 * s)


x_initial = analytical_1(s_grid)
x_initial_internal = x_initial[1:-1]

# Get operator
A = forward_heat_operator(N_int, T)

# We only observe some of the terminal conditions
obs_min = np.pi / 4
obs_max = 3 / 4 * np.pi
obs_mask = (obs_min < s_internal) & (s_internal < obs_max)

y_full = A @ x_initial_internal
y_obs = y_full[obs_mask]

A_obs = A[obs_mask, :]  # Observed rows only

# ---
M = A_obs.shape[0]  # Number of observations
I_obs = np.eye(M)
I_state = np.eye(N_int)

posterior_mean_obs = (
    gamma**2
```

```
    * A_obs.T
    @ np.linalg.inv(gamma**2 * A_obs @ A_obs.T + sigma**2 * I_obs)
    @ y_obs
)
posterior_covariance_obs = (
    gamma**2 * I_state
    + gamma**4
    * A_obs.T
    @ np.linalg.inv(gamma**2 * A_obs @ A_obs.T + sigma**2 * I_obs)
    @ A_obs
)
posterior_variance_obs = np.diag(posterior_covariance_obs)

# Sample posterior
N_simulations = 50
np.random.seed(42)
x_samples_obs = np.random.multivariate_normal(
    posterior_mean_obs, posterior_covariance_obs, size=N_simulations
)

sample_mean_obs = np.mean(x_samples_obs, axis=0)
sample_variance_obs = np.var(x_samples_obs, axis=0)

fig, ax = plt.subplots(figsize=(PAPER_WIDTH_IN, 3))
ax.plot(
    s_internal,
    x_initial_internal,
    color="black",
    linewidth=3.0,
    label="Initial condition",
)
ax.plot(s_internal, sample_mean_obs, color="C0", lw=2.0, label="Sample mean")
ax.plot(s_internal, posterior_mean_obs, color="C1", ls="--", label="Posterior mean")

# Plot 95% credible intervals
sample_upper_bound = sample_mean_obs + 1.96 * np.sqrt(sample_variance_obs)
sample_lower_bound = sample_mean_obs - 1.96 * np.sqrt(sample_variance_obs)
posterior_upper_bound = posterior_mean_obs + 1.96 * np.sqrt(posterior_variance_obs)
posterior_lower_bound = posterior_mean_obs - 1.96 * np.sqrt(posterior_variance_obs)
ax.fill_between(
    s_internal, sample_lower_bound, sample_upper_bound, color="C0", alpha=0.3
)
ax.fill_between(
    s_internal, posterior_lower_bound, posterior_upper_bound, color="C1", alpha=0.3
)

# Plot observation region
ax.axvspan(obs_min, obs_max, color="gray", alpha=0.2, label="Observation region")

legend_elements = [
```

```
        mpl.patches.Patch(facecolor="gray", alpha=0.2),
        mpl.lines.Line2D([0], [0], color="black", linewidth=3.0),
        (
            mpl.lines.Line2D([0], [0], color="C0", linewidth=2.0, ls="-"),
            mpl.patches.Patch(facecolor="C0", alpha=0.3),
        ),
        (
            mpl.lines.Line2D([0], [0], color="C1", ls="--"),
            mpl.patches.Patch(facecolor="C1", alpha=0.3),
        ),
    ]

    legend_labels = [
        "Observed",
        "$x_\\mathrm{true}$",
        "$x_\\mathrm{sample}$\n(95% CI)",
        "$x_\\mathrm{post}$\n(95% CI)",
    ]

    ax.legend(legend_elements, legend_labels, loc="center left", bbox_to_anchor=(1,
0.5))

    ax.set_xlabel("Spatial coordinate, $s$")
    ax.set_ylabel("Initial Temperature, $x(s)$")

    fig.suptitle(
        "Analytical and Sampled Bayesian Estimation of Initial Temperature",
        fontsize=12,
        y=0.92,
    )
    fig.tight_layout()
    plt.savefig(OUTPUT_DIR / "1_c.png", dpi=EXPORT_DPI, bbox_inches="tight")

    # ---
    # Double plot

    # Parameters
    N = 128
    N_int = N - 2
    T = 0.1
    a = 0
    b = np.pi

    gamma = 1e-1
    sigma = 1e-3

    # Get spatial discretisation
    s_grid = np.linspace(a, b, N, endpoint=True)
    s_internal = s_grid[1:-1]  # Internal points only


```

```python
def analytical_1(s: npt.NDArray) -> npt.NDArray:
    return np.sin(4 * s)


x_initial = analytical_1(s_grid)
x_initial_internal = x_initial[1:-1]

np.random.seed(42)

fig, (ax_long, ax_short) = plt.subplots(
    1, 2, figsize=(PAPER_WIDTH_IN, 3), sharey=True, sharex=True
)
for T, ax in [(0.1, ax_long), (0.005, ax_short)]:
    # Get operator
    A = forward_heat_operator(N_int, T)

    # We only observe some of the terminal conditions
    obs_min = np.pi / 4
    obs_max = 3 / 4 * np.pi
    obs_mask = (obs_min < s_internal) & (s_internal < obs_max)

    y_full = A @ x_initial_internal
    y_obs = y_full[obs_mask]

    A_obs = A[obs_mask, :]  # Observed rows only

    M = A_obs.shape[0]  # Number of observations
    I_obs = np.eye(M)
    I_state = np.eye(N_int)

    posterior_mean_obs = (
        gamma**2
        * A_obs.T
        @ np.linalg.inv(gamma**2 * A_obs @ A_obs.T + sigma**2 * I_obs)
        @ y_obs
    )
    posterior_covariance_obs = (
        gamma**2 * I_state
        + gamma**4
        * A_obs.T
        @ np.linalg.inv(gamma**2 * A_obs @ A_obs.T + sigma**2 * I_obs)
        @ A_obs
    )
    posterior_variance_obs = np.diag(posterior_covariance_obs)

    # Sample posterior
    N_simulations = 50
    x_samples_obs = np.random.multivariate_normal(
        posterior_mean_obs, posterior_covariance_obs, size=N_simulations
    )
```

```
193
194     sample_mean_obs = np.mean(x_samples_obs, axis=0)
195     sample_variance_obs = np.var(x_samples_obs, axis=0)
196
197     ax.plot(
198         s_internal,
199         x_initial_internal,
200         color="black",
201         linewidth=1.5,
202         label="Initial condition",
203     )
204     ax.plot(s_internal, sample_mean_obs, color="C0", lw=2.0, label="Sample mean")
205     ax.plot(s_internal, posterior_mean_obs, color="C1", ls="--", label="Posterior
        mean")
206
207     # Plot 95% credible intervals
208     sample_upper_bound = sample_mean_obs + 1.96 * np.sqrt(sample_variance_obs)
209     sample_lower_bound = sample_mean_obs - 1.96 * np.sqrt(sample_variance_obs)
210     posterior_upper_bound = posterior_mean_obs + 1.96 *
        np.sqrt(posterior_variance_obs)
211     posterior_lower_bound = posterior_mean_obs - 1.96 *
        np.sqrt(posterior_variance_obs)
212     ax.fill_between(
213         s_internal, sample_lower_bound, sample_upper_bound, color="C0", alpha=0.3
214     )
215     ax.fill_between(
216         s_internal, posterior_lower_bound, posterior_upper_bound, color="C1",
            alpha=0.3
217     )
218
219     # Plot observation region
220     ax.axvspan(obs_min, obs_max, color="gray", alpha=0.2, label="Observation
        region")
221
222     ax.set_title(f"$T = {T}$")
223
224 legend_elements = [
225     mpl.patches.Patch(facecolor="gray", alpha=0.2),
226     mpl.lines.Line2D([0], [0], color="black", linewidth=1.5),
227     (
228         mpl.lines.Line2D([0], [0], color="C0", linewidth=2.0, ls="-"),
229         mpl.patches.Patch(facecolor="C0", alpha=0.3),
230     ),
231     (
232         mpl.lines.Line2D([0], [0], color="C1", ls="--"),
233         mpl.patches.Patch(facecolor="C1", alpha=0.3),
234     ),
235 ]
236
237 legend_labels = [
238     "Observed",
239     "$x_\\mathrm{true}$",
```

```
    "$x_\\mathrm{sample}$\n(95% CI)",
    "$x_\\mathrm{post}$\n(95% CI)",
]

ax.legend(legend_elements, legend_labels, loc="center left", bbox_to_anchor=(1, 0.5))

ax_short.set_xlabel("Spatial coordinate, $s$")
ax_long.set_xlabel("Spatial coordinate, $s$")
ax_long.set_ylabel("Initial Temperature, $x(s)$")

fig.suptitle(
    "Analytical and Sampled Bayesian Estimation of Initial Temperature",
    fontsize=12,
    y=0.92,
)
fig.tight_layout()
plt.savefig(OUTPUT_DIR / "1_c_double.png", dpi=EXPORT_DPI, bbox_inches="tight")
```

A.4) Code for Section 2.c

Python

```
# Abel Problem
N = 1024
a = 0
b = 1
h = (b - a) / N
s = a + (np.arange(N) + 0.5) * h
t = s + h / 2  # Shift t values to prevent singularity at t=s

s_mesh, t_mesh = np.meshgrid(s, t)

A = np.zeros((N, N))
indices = s_mesh < t_mesh  # Select according to integral
A[indices] = h / np.sqrt((t_mesh[indices] - s_mesh[indices]))


def x_2_func(s):
    return s ** (3 / 4) - 1


def grad_x_2_func(s):
    return (3 / 4) * s ** (-1 / 4)


x_true = x_2_func(s)
grad_x_true = grad_x_2_func(s)
y_true = A @ x_true

# Add noise, 10%
target_noise_level = 0.10
unscaled_noise = np.random.randn(*y_true.shape)
unscaled_noise_level = np.linalg.norm(unscaled_noise)
```

```
noise_coeff = target_noise_level * np.linalg.norm(y_true) / unscaled_noise_level
noise = noise_coeff * unscaled_noise
noise_level = np.linalg.norm(noise)
print(f"Achieved noise level: {noise_level}")
y_noisy = y_true + noise

# Plot noisy measurements
plt.figure(figsize=(PAPER_WIDTH_IN, 2.2))
plt.plot(t, y_noisy, label="$y^δ(t)$", lw=1)
plt.plot(t, y_true, label="$y(t)$", lw=1.5, ls="--")
plt.title("Noisy Measurements for Abel Problem given $x(s) = s^2$")
plt.xlabel("Temporal Coordinate, $t$")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_a.png", dpi=600, bbox_inches="tight")
display(plt.gcf())

plt.gcf().set_size_inches(PAPER_WIDTH_IN, 3.5)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "2_a_large.png", dpi=600, bbox_inches="tight")

# ---


def create_differentiation_matrix(n: int, h: float) -> npt.NDArray:
    D = np.zeros((n, n))

    # Use forward difference for the first point
    D[0, 0] = -1.0 / h
    D[0, 1] = 1.0 / h

    # Central difference for interior points
    for i in range(1, n - 1):
        D[i, i - 1] = -0.5 / h
        D[i, i + 1] = 0.5 / h

    # Backward difference for the last point
    D[n - 1, n - 2] = -1.0 / h
    D[n - 1, n - 1] = 1.0 / h

    return D


D = create_differentiation_matrix(N, h)

grad_x_estimate = D @ x_true
plt.plot(s, grad_x_true, label="True Gradient", lw=1.5)
plt.plot(s, grad_x_estimate, label="Estimated Gradient", lw=1)
plt.figure()
plt.plot(s, np.abs(grad_x_true - grad_x_estimate), label="Absolute Error", lw=1)
```

```
82
83  # ---
84
85
86  def tikhonov_ordinary(alpha: float) -> npt.NDArray:
87      n = A.shape[1]
88      x_alpha = np.linalg.inv(A.T @ A + alpha * np.eye(n)) @ A.T @ y_noisy
89      return x_alpha
90
91
92  def tikhonov_generalised_smoothness(alpha: float) -> npt.NDArray:
93      # TODO: Derive in report
94      n = A.shape[1]
95      x_alpha = np.linalg.inv(A.T @ A + alpha * (np.eye(n) + D.T @ D)) @ A.T @ y_noisy
96      return x_alpha
97
98
99  def F_1(alpha, x_alpha):
100     return np.linalg.norm(A @ x_alpha - y_noisy)
101
102
103 alphas = np.linspace(0, 0.2, 200)
104 data = []
105 for alpha in tqdm(alphas):
106     x_alpha_ordinary = tikhonov_ordinary(alpha)
107     F_1_val_ordinary = F_1(alpha, x_alpha_ordinary)
108     x_diff_ordinary = x_alpha_ordinary - x_true
109
110     x_alpha_generalised = tikhonov_generalised_smoothness(alpha)
111     F_1_val_generalised = F_1(alpha, x_alpha_generalised)
112     x_diff_generalised = x_alpha_generalised - x_true
113
114     data.append(
115         {
116             "alpha": alpha,
117             "F_1_ordinary": F_1_val_ordinary,
118             "x_alpha_ordinary": x_alpha_ordinary,
119             "x_diff_ordinary": x_diff_ordinary,
120             "x_diff_ordinary_L2": np.linalg.norm(x_diff_ordinary),
121             "F_1_generalised": F_1_val_generalised,
122             "x_alpha_generalised": x_alpha_generalised,
123             "x_diff_generalised": x_diff_generalised,
124             "x_diff_generalised_L2": np.linalg.norm(x_diff_generalised),
125         }
126     )
127
128 df = pd.DataFrame(data)
129
130 df_2c = df.copy()
131
```

```
# ---
# Safety factor, referred to as nu in lecture notes
safety_factor = 1.05

# Find optimal alpha for ordinary Tikhonov
df_2c["ordinary_search_target"] = df_2c["F_1_ordinary"] - safety_factor *
noise_level
idx_ordinary = df_2c["ordinary_search_target"].abs().idxmin()
df_2c["generalised_search_target"] = (
    df_2c["F_1_generalised"] - safety_factor * noise_level
)
idx_generalised = df_2c["generalised_search_target"].abs().idxmin()
idx_ordinary_true = df_2c["x_diff_ordinary_L2"].abs().idxmin()
idx_generalised_true = df_2c["x_diff_generalised_L2"].abs().idxmin()
print(
    f"Optimal alpha (ordinary): Estimate: {df_2c.iloc[idx_ordinary]['alpha']}, True
    min: {df_2c.iloc[idx_ordinary_true]['alpha']}"
)
print(
    f"Optimal alpha (generalised): Estimate: {df_2c.iloc[idx_generalised]['alpha']},
    True min: {df_2c.iloc[idx_generalised_true]['alpha']}"
)

ordinary_estimate_alpha = df_2c.iloc[idx_ordinary]["alpha"]
ordinary_estimate_residual = df_2c.iloc[idx_ordinary]["F_1_ordinary"]
ordinary_estimate_error = df_2c.iloc[idx_ordinary]["x_diff_ordinary_L2"]
generalised_estimate_alpha = df_2c.iloc[idx_generalised]["alpha"]
generalised_estimate_residual = df_2c.iloc[idx_generalised]["F_1_generalised"]
generalised_estimate_error = df_2c.iloc[idx_generalised]["x_diff_generalised_L2"]
ordinary_true_alpha = df_2c.iloc[idx_ordinary_true]["alpha"]
ordinary_true_residual = df_2c.iloc[idx_ordinary_true]["F_1_ordinary"]
ordinary_true_error = df_2c.iloc[idx_ordinary_true]["x_diff_ordinary_L2"]
generalised_true_alpha = df_2c.iloc[idx_generalised_true]["alpha"]
generalised_true_residual = df_2c.iloc[idx_generalised_true]["F_1_generalised"]
generalised_true_error = df_2c.iloc[idx_generalised_true]["x_diff_generalised_L2"]

ordinary_best_x = df_2c.iloc[idx_ordinary]["x_alpha_ordinary"]
ordinary_best_alpha = df_2c.iloc[idx_ordinary]["alpha"]

# ---
# Discrepancy plot
plt.figure(figsize=(PAPER_WIDTH_IN, 2))
sns.lineplot(data=df, x="alpha", y="F_1_ordinary", label="$||Ax_α- yδ||$
(ordinary)")
# sns.lineplot(data=df, x="alpha", y="F_1_generalised")

plt.axhline(noise_level, color="black", ls=":", label="Noise level")
plt.axhline(
    noise_level * safety_factor,
    color="black",
    ls="--",
```

```
    label="Noise level (safety factor)",
)

plt.plot(
    ordinary_estimate_alpha,
    ordinary_estimate_residual,
    "o",
    color="C0",
    markersize=7,
    label="$α_\\mathrm{DP}$",
)
plt.plot(
    ordinary_true_alpha,
    ordinary_true_residual,
    "*",
    color="tab:red",
    markersize=9,
    label="$α_\\mathrm{optimal}$",
)
plt.legend()

plt.title("Discrepancy Principle for Ordinary Tikhonov Regularisation")
plt.xlabel("Regualarisation Parameter, $α$")
plt.ylabel("Residual Norm, $||Ax_α - y^δ||$")

plt.figure()
sns.lineplot(data=df, x="alpha", y="x_diff_ordinary_L2")
plt.plot(
    ordinary_estimate_alpha,
    ordinary_estimate_error,
    "o",
    color="C0",
    markersize=7,
    label="$α_\\mathrm{DP}$",
)
plt.plot(
    ordinary_true_alpha,
    ordinary_true_error,
    "*",
    color="tab:red",
    markersize=9,
    label="$α_\\mathrm{optimal}$",
)
# sns.lineplot(data=df, x="alpha", y="x_diff_generalised_L2")
plt.semilogy()

# ---
# Discrepancy plot
fig, (ax_resid, ax_error) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, 2.5))

```

```
sns.lineplot(ax=ax_resid, data=df_2c, x="alpha", y="F_1_ordinary")
# sns.lineplot(data=df_2c, x="alpha", y="F_1_generalised")

ax_resid.axhline(noise_level, color="black", ls=":", label="Noise level, $δ$")
ax_resid.axhline(
    noise_level * safety_factor, color="black", ls="--", label="Augmented level, $νδ$"
)

ax_resid.plot(
    ordinary_estimate_alpha,
    ordinary_estimate_residual,
    "o",
    color="C1",
    markersize=7,
    label="$α_\\mathrm{DP}$",
)
ax_resid.plot(
    ordinary_true_alpha,
    ordinary_true_residual,
    "*",
    color="C2",
    markersize=9,
    label="$α_\\mathrm{optimal}$",
)

ax_resid.set_xlabel("Regualarisation Parameter, $α$")
ax_resid.set_ylabel("Residual Norm, $||Ax_α - y^δ||$")

sns.lineplot(ax=ax_error, data=df_2c, x="alpha", y="x_diff_ordinary_L2", color="C0")
ax_error.plot(
    ordinary_estimate_alpha, ordinary_estimate_error, "o", color="C1", markersize=7
)
ax_error.plot(ordinary_true_alpha, ordinary_true_error, "*", color="C2", markersize=9)
# sns.lineplot(data=df_2c, x="alpha", y="x_diff_generalised_L2")
ax_error.set_yscale("log")
ax_error.grid(axis="y", which="both")
ax_error.set_xlabel("Regualarisation Parameter, $α$")
ax_error.set_ylabel("Error Norm, $||x_α - x||$")

# Get legends and handles from both axes
handles1, labels1 = ax_resid.get_legend_handles_labels()
handles2, labels2 = ax_error.get_legend_handles_labels()

handles = [handles1[0]] + handles2 + handles1[1:]
labels = [labels1[0]] + labels2 + labels1[1:]

ax_error.legend(handles, labels)
ax_resid.legend().remove()

```

```
fig.suptitle(
    "Discrepancy Principle for Ordinary Tikhonov Regularisation", fontsize=12, y=0.92
)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "2_c.png", dpi=600, bbox_inches="tight")

# ---
x_alpha_optim_ordinary = df_2c.iloc[idx_ordinary]["x_alpha_ordinary"]
x_alpha_optim_generalised = df_2c.iloc[idx_generalised]["x_alpha_generalised"]
plt.plot(s, x_true, label="True solution", lw=2.0)
plt.plot(s, x_alpha_optim_ordinary, label="Ordinary Tikhonov", lw=1.5)
plt.plot(s, x_alpha_optim_generalised, label="Generalised Tikhonov", lw=1.5)
plt.title("Tikhonov Regularisation Solutions for Abel Problem")
plt.legend()
```

A.5) Code for Section 2.d

Python

```


def tikhonov_ordinary(alpha: float) -> npt.NDArray:
    n = A.shape[1]
    x_alpha = np.linalg.inv(A.T @ A + alpha * np.eye(n)) @ A.T @ y_noisy
    return x_alpha


def tikhonov_generalised_smoothness(alpha: float) -> npt.NDArray:
    # TODO: Derive in report
    n = A.shape[1]
    x_alpha = np.linalg.inv(A.T @ A + alpha * (np.eye(n) + D.T @ D)) @ A.T @ y_noisy
    return x_alpha


def F_1(alpha, x_alpha):
    return np.linalg.norm(A @ x_alpha - y_noisy)


alphas = np.linspace(0, 0.015, 200)
data = []
for alpha in tqdm(alphas):
    x_alpha_ordinary = tikhonov_ordinary(alpha)
    F_1_val_ordinary = F_1(alpha, x_alpha_ordinary)
    x_diff_ordinary = x_alpha_ordinary - x_true

    x_alpha_generalised = tikhonov_generalised_smoothness(alpha)
    F_1_val_generalised = F_1(alpha, x_alpha_generalised)
    x_diff_generalised = x_alpha_generalised - x_true

    data.append(
        {
            "alpha": alpha,
            "F_1_ordinary": F_1_val_ordinary,
```

```
            "x_alpha_ordinary": x_alpha_ordinary,
            "x_diff_ordinary": x_diff_ordinary,
            "x_diff_ordinary_L2": np.linalg.norm(x_diff_ordinary),
            "F_1_generalised": F_1_val_generalised,
            "x_alpha_generalised": x_alpha_generalised,
            "x_diff_generalised": x_diff_generalised,
            "x_diff_generalised_L2": np.linalg.norm(x_diff_generalised),
        }
    )

df = pd.DataFrame(data)
df_2d = df.copy()

# ---
# Find optimal alpha for ordinary Tikhonov
df_2d["ordinary_search_target"] = df_2d["F_1_ordinary"] - safety_factor *
noise_level
idx_ordinary = df_2d["ordinary_search_target"].abs().idxmin()
df_2d["generalised_search_target"] = (
    df_2d["F_1_generalised"] - safety_factor * noise_level
)
idx_generalised = df_2d["generalised_search_target"].abs().idxmin()
idx_ordinary_true = df_2d["x_diff_ordinary_L2"].abs().idxmin()
idx_generalised_true = df_2d["x_diff_generalised_L2"].abs().idxmin()
print(
    f"Optimal alpha (ordinary): Estimate: {df_2d.iloc[idx_ordinary]['alpha']}, True
    min: {df_2d.iloc[idx_ordinary_true]['alpha']}"
)
print(
    f"Optimal alpha (generalised): Estimate: {df_2d.iloc[idx_generalised]['alpha']},
    True min: {df_2d.iloc[idx_generalised_true]['alpha']}"
)

ordinary_estimate_alpha = df_2d.iloc[idx_ordinary]["alpha"]
ordinary_estimate_residual = df_2d.iloc[idx_ordinary]["F_1_ordinary"]
ordinary_estimate_error = df_2d.iloc[idx_ordinary]["x_diff_ordinary_L2"]
generalised_estimate_alpha = df_2d.iloc[idx_generalised]["alpha"]
generalised_estimate_residual = df_2d.iloc[idx_generalised]["F_1_generalised"]
generalised_estimate_error = df_2d.iloc[idx_generalised]["x_diff_generalised_L2"]
ordinary_true_alpha = df_2d.iloc[idx_ordinary_true]["alpha"]
ordinary_true_residual = df_2d.iloc[idx_ordinary_true]["F_1_ordinary"]
ordinary_true_error = df_2d.iloc[idx_ordinary_true]["x_diff_ordinary_L2"]
generalised_true_alpha = df_2d.iloc[idx_generalised_true]["alpha"]
generalised_true_residual = df_2d.iloc[idx_generalised_true]["F_1_generalised"]
generalised_true_error = df_2d.iloc[idx_generalised_true]["x_diff_generalised_L2"]

generalised_best_x = df_2d.iloc[idx_generalised]["x_alpha_generalised"]
generalised_best_alpha = df_2d.iloc[idx_generalised]["alpha"]

# ---
# Discrepancy plot
```

```
fig, (ax_resid, ax_error) = plt.subplots(1, 2, figsize=(PAPER_WIDTH_IN, 2.5))

sns.lineplot(ax=ax_resid, data=df_2d, x="alpha", y="F_1_generalised")

ax_resid.axhline(noise_level, color="black", ls=":", label="Noise level, $δ$")
ax_resid.axhline(
    noise_level * safety_factor, color="black", ls="--", label="Augmented level, $νδ$"
)

ax_resid.plot(
    generalised_estimate_alpha,
    generalised_estimate_residual,
    "o",
    color="C1",
    markersize=7,
    label="$α_\\mathrm{DP}$",
)
ax_resid.plot(
    generalised_true_alpha,
    generalised_true_residual,
    "*",
    color="C2",
    markersize=9,
    label="$α_\\mathrm{optimal}$",
)

ax_resid.set_xlabel("Regualarisation Parameter, $α$")
ax_resid.set_ylabel("Residual Norm, $||Ax_α - y^δ||$")

sns.lineplot(ax=ax_error, data=df_2d, x="alpha", y="x_diff_generalised_L2", color="C0")
ax_error.plot(
    generalised_estimate_alpha,
    generalised_estimate_error,
    "o",
    color="C1",
    markersize=7,
)
ax_error.plot(
    generalised_true_alpha, generalised_true_error, "*", color="C2", markersize=9
)
# sns.lineplot(data=dfs, x="alpha", y="x_diff_generalised_L2")
ax_error.set_yscale("log")
ax_error.grid(axis="y", which="both")
ax_error.set_xlabel("Regualarisation Parameter, $α$")
ax_error.set_ylabel("Error Norm, $||x_α - x||$")

# Get legends and handles from both axes
handles1, labels1 = ax_resid.get_legend_handles_labels()
handles2, labels2 = ax_error.get_legend_handles_labels()
```

```
132
133 handles = [handles1[0]] + handles2 + handles1[1:]
134 labels = [labels1[0]] + labels2 + labels1[1:]
135
136 ax_error.legend(handles, labels)
137 ax_resid.legend().remove()
138
139 fig.suptitle(
140     "Discrepancy Principle for Generalised Tikhonov Regularisation", fontsize=12,
    y=0.92
141 )
142 fig.tight_layout()
143 fig.savefig(OUTPUT_DIR / "2_d.png", dpi=600, bbox_inches="tight")
144
145 # ---
146 plt.figure(figsize=(PAPER_WIDTH_IN, 2.5))
147 plt.plot(
148     s,
149     ordinary_best_x,
150     label=f"Ordinary Tikhonov, $x_{{α={ordinary_best_alpha:.4f}}}$",
151     lw=1,
152 )
153 plt.plot(
154     s,
155     generalised_best_x,
156     label=f"Generalised Tikhonov, $x_{{α={generalised_best_alpha:.4f}}}$",
157     lw=2,
158 )
159
160 plt.plot(s, x_true, label="True Solution, $x$", lw=2.0, ls="--", color="black")
161 plt.legend()
162
163 plt.xlabel("Spatial Coordinate, $s$")
164 plt.ylabel("Solution, $x(s)$")
165 plt.title("Tikhonov Regularisation Solutions for Abel Problem")
166
167 plt.tight_layout()
168 plt.savefig(OUTPUT_DIR / "2_d_solutions.png", dpi=600, bbox_inches="tight")
```

A.6) Google Gemini 2.5 Pro Review Comments

Note: This uses a generic *reviewer* Gem.

User

Homework2
PDF

HW2
PDF

Gemini

Here is the review of the assignment based on the provided prompt and student submission.

1. Executive Summary

Overall Assessment: The report demonstrates a solid understanding of the theoretical underpinnings of Bayesian inversion and Tikhonov regularization. The derivations in Section 1 are mathematically sound, and the numerical experiments in Section 2 generally align with the prompt's objectives. However, the work is significantly undermined by a failure to use the specific parameters provided in the problem statement for Problem 1. This alters the noise characteristics and prior variance, meaning the results presented—while internally consistent—do not solve the specific problem posed.

Priority Revisions:

- **Parameter Discrepancy (Critical):** In Problem 1, you utilized $\gamma = 10^{-1}$ and $\sigma = 10^{-3}$. The prompt explicitly specifies $\gamma = 1.0$ and $\sigma = 10^{-2}$. This order-of-magnitude difference alters the signal-to-noise ratio and the regularization strength (α), rendering the specific numerical results incorrect relative to the prompt.
- **Sample Size:** In Problem 1(b), the prompt requests "say 100" samples. You provided plots for $N = 5$ and $N = 500$. While illustrative, you should include the requested sample size or explicitly justify the deviation.
- **Interpretation of Discrepancy Principle:** In Problem 2(d), you note that the safety factor $\nu = 1.05$ negatively impacts the result. You must explain *why* this occurs in the context of the specific noise realization or the smoothness of the solution, rather than just noting the observation.

---

2. Detailed Section-by-Section Analysis

1) 1D Heat Problem

1.a) Deriving the Ordinary Tikhonov Solution

- **Completeness:** You successfully derive the Tikhonov form and demonstrate the covariance inequality.
- **Correctness:**
  - The derivation of the posterior mean and covariance using the "completing the square" method and the Woodbury matrix identity is mathematically correct.
  - The proof that $\Gamma_{post} < \gamma^2 I$ is logically sound, relying correctly on the positive definiteness of the inverse term.
- **Clarity:** The steps are well-structured and easy to follow. The transition from equation (10) to (12) clearly connects the Bayesian result to the deterministic Tikhonov regularizer.

1.b) Analytical and Sampled Bayesian Estimation

- **Completeness:** You computed sample statistics and plotted the required curves (mean, posterior mean, CI). However, as noted in the Executive Summary, you did not use the requested sample size ($N = 100$).
- **Correctness:**
  - **Parameter Error:** The results in Figure 1 generate a posterior based on $\sigma = 10^{-3}$ and $\gamma = 0.1$. The prompt specifies $\sigma = 10^{-2}$ and $\gamma = 1$. Consequently, your posterior is much tighter (lower variance) than the problem intends.
  - The implementation of the sampling and confidence intervals appears algorithmically correct based on the figures provided.
- **Clarity:** The discussion regarding finite sampling effects is clear and well-illustrated by the comparison between $N = 5$ and $N = 500$.

1.c) Reduced Observational Region

- **Completeness:** You analyzed the reduced observation region. You included an unrequested analysis of a different terminal time ($T = 0.005$), which, while interesting, distracts slightly from the direct comparison of the observation operator A versus its submatrix.
- **Correctness:**
  - The observation that uncertainty increases in unobserved regions and reverts to the prior is correct.
  - Your claim that "shorter terminal times ordinarily resulting in a less ill-posed problem" is generally true for heat flow (less smoothing), but your analysis of $T = 0.005$ showing the estimate reverting to the prior is a consequence of the *lack of information propagation* to the unobserved region in short time, not necessarily ill-posedness in the observed region. Be careful with this distinction.
- **Clarity:** The comparison in Figure 2 is effective, though the discussion should focus more strictly on the "missing information" aspect required by the prompt's hint.

2) Abel Problem and H^1-Regularisation

2.a) Show that $x(s) \in H^1(0,1)$

- **Completeness:** Addressed fully.
- **Correctness:**
  - The integration of $x(s)^2$ is correct.
  - The integration of the derivative $(x'(s))^2 \propto s^{-1/2}$ is correct ($9/8 < \infty$).
- **Clarity:** The breakdown of the calculation is precise and easy to verify.

2.b) Uniqueness and Existence

- **Completeness:** Addressed fully.
- **Correctness:**
  - The use of the Weierstrass Extreme Value Theorem relying on coercivity is the standard and correct approach for existence.
  - The proof of uniqueness via the strict convexity of the objective function (positive definite Hessian) is correct. You correctly identified that adding αI ensures the Hessian is strictly positive definite even if $A^T A$

or $D^T D$ are singular.

- **Clarity:** Good use of definitions and clear logical progression.

2.c) Ordinary Tikhonov and Discrepancy Principle

- **Completeness:** You solved the problem and plotted the error relative to the optimal α.
- **Correctness:**
  - The application of the discrepancy principle appears correct based on the intersection shown in Figure 3.
  - The "Optimal" marker in Figure 3 aligns with the minimum of the error norm, demonstrating a correct validation method.
- **Clarity:** Figure 3 is highly legible. The dual-axis/dual-panel presentation separates the residual analysis from the error analysis effectively.

2.d) Generalised Tikhonov and Discrepancy Principle

- **Completeness:** You compared the generalized solution to the ordinary one.
- **Correctness:**
  - The discretization of D (Equation 30) is standard.
  - You identified that the safety factor $\nu = 1.05$ caused over-regularization in this specific instance.
- **Clarity:** The comparison in Figure 5 clearly demonstrates the advantage of H^1 regularization for smooth functions.

---

3. Analysis of Missed Insights

- **Singularity and Regularization:** In Problem 2, the function $x(s) = s^{3/4} - 1$ has a derivative $x'(s) \propto s^{-1/4}$. While this is in L^2, it approaches infinity as $s \to 0$. Regularization involving $||Dx||_2^2$ penalizes large derivatives. You should have analyzed how the Generalized Tikhonov solution behaves specifically near $s = 0$ compared to the Ordinary Tikhonov solution. Does the penalization of the derivative cause a larger bias near the singularity? Figure 5 suggests a deviation near $s = 0$ for the generalized case, but the text does not discuss this specific trade-off.
- **Correlation Length in 1(c):** When discussing the reduced observation region, you mention the estimate reverts to the prior. A deeper insight would be to discuss the *correlation length* implied by the posterior covariance. In the heat equation, information diffuses; the distance into the unobserved region that the posterior variance remains lower than the prior variance is a direct measure of the "information horizon" at time T.
- **Impact of Safety Factor ν:** In 2(d), you noted $\nu = 1.05$ hurt performance. A missed insight is *why*. The Discrepancy Principle is known to over-smooth when the noise level is overestimated or when the solution is extremely smooth. Since the true solution is very smooth (save for the singularity), the "optimal" α allows for very little regularization. The safety factor forces higher α, pushing the solution away from this optimum.

---

4. Formatting and Notation Review

- **Citation Format:** The report adheres to the requested numeric citation style.
- **Figures:** The figures are well-labeled with clear legends and captions.
- **Math Notation:** Notation is consistent (e.g., Γ_{post}, $\overline{x}$).
- **Structure:** The report structure mirrors the assignment structure well, aiding readability.

- **Code:** The code is referenced as being in the appendix (though not reviewed here), satisfying the submission requirement.

User

Please also find any grammatical errors that may be present. Feel free to be pedantic

Gemini

Here is a pedantic review of grammatical, typographical, and stylistic errors found in the report.

1. Typographical and Orthographical Errors

- **Source 22:** "Where is a realisation of the prior..."
  - **Error:** Missing subject.
  - **Correction:** "Where x is a realisation..." or "Where X is a realisation...".
- **Source 39:** Equation derivation lines.
  - **Error:** The terms "y_{dan}" and "y_{dab}" appear in the expansion. These appear to be severe typos for y_{obs}.
  - **Correction:** Replace with y_{obs}.
- **Source 73:** "...we seek to use the the Weierstrass Extreme Value Theorem..."
  - **Error:** Repetition of the word "the".
  - **Correction:** "...use **the** Weierstrass...".
- **Source 74:** Equation (14).
  - **Error:** "$||z|_2^2$"
  - **Correction:** Missing the closing vertical bar: "$||z||_2^2$".
- **Source 88, 106, 137:** Figure Captions.
  - **Error:** Multiple misspellings of the subscript "obs" in the figure captions and text, specifically "y_{otxs}" and "y_{ohs}".
  - **Correction:** Ensure consistent notation: y_{obs}.
- **Source 162:** "...we seek to use the the Weierstrass..."
  - **Error:** Repetition of the word "the" (Note: This error appears in both the derivation reference in section 1.a and section 2.b, suggesting a copy-paste error).
  - **Correction:** "...use **the** Weierstrass...".
- **Source 208:** "...we also display the error norma..."
  - **Error:** "norma".
  - **Correction:** "**norm**".
- **Source 209:** "...regularisation parameter very close the the optimal value."

- **Error:** Missing preposition "to" and repetition of "the".
- **Correction:** "...very close **to the** optimal value".

2. Grammatical and Syntactical Errors

- **Source 106:** "...prior standard deviation $\gamma = 10^{-1}$ The black line..."
  - **Error:** Run-on sentence; missing punctuation after the mathematical expression.
  - **Correction:** "...$\gamma = 10^{-1}$. The black line...".
- **Source 137:** "...prior standard deviation $\gamma = 10^{-1}$ Only the region..."
  - **Error:** Run-on sentence; missing punctuation.
  - **Correction:** "...$\gamma = 10^{-1}$. Only the region...".
- **Source 167:** "...to conclude that a global minimiser exists on arbitrarily chosen sublevel set S_c."
  - **Error:** Missing article.
  - **Correction:** "...exists on **an** arbitrarily chosen sublevel set...".
- **Source 246:** "...appears negatively impact the performance..."
  - **Error:** Missing infinitive marker "to".
  - **Correction:** "...appears **to** negatively impact...".

3. Stylistic and Consistency Issues

- **Spelling Consistency (British vs. American):**
  - The assignment prompt uses American English spelling (e.g., "modeled" , "Regularization" , "Discretize").
  - Your report predominantly uses British English (e.g., "modelled" , "Regularisation" , "discretised" , "realisation" , "minimiser"). While acceptable, it is generally preferred to match the dialect of the publication or prompt you are addressing.
- **Repetitive Phrasing:**
  - The phrase "As such" is used excessively as a transition (Sources 23, 26, 65, 77, 155, 188, 248). Consider varying your transitional phrases (e.g., "Consequently," "Therefore," "Thus," "Hence").
- **Notation Consistency:**
  - In Source 193, you utilize the symbol `\top` ($A^\top$) for the transpose, whereas in the majority of the text (e.g., Source 41, 48, 58), you use `T` (A^T). Select one notation and adhere to it strictly.

4. Formatting Artifacts

- **Source 39:** "$= exp(-\frac{1}{2})x^T$..."

- **Observation:** The closing parenthesis for the exponential function appears prematurely or is syntactically incorrect in the LaTeX source. It reads as if you are multiplying $e^{-1/2}$ by the quadratic form, rather than the quadratic form being the argument of the exponential.

- **Source 54:** "[2, Eq. 2.18]"
  - **Observation:** In the text extraction, this citation appears broken across lines. Ensure non-breaking spaces are used in LaTeX (e.g., `Eq.~2.18`) to prevent valid references from being orphaned.

Bibliography

[1] K. B. Petersen and M. S. Pedersen, “The Matrix Cookbook.”

[2] A. Kirsch, *An Introduction to the Mathematical Theory of Inverse Problems*, vol. 120. in Applied Mathematical Sciences, vol. 120. Cham: Springer International Publishing, 2021. doi: 10.1007/978-3-030-63343-1.

[3] M. Andersen, *DTU Course 02611, L1: Introduction*. 2025. doi: 10.1002/9781119809180.

[4] P. C. Hansen, *Discrete Inverse Problems: Insight and Algorithms*, no. 7. in Fundamentals of Algorithms. Philadelphia: Society for Industrial and Applied Mathematics, 2010.